

AttestFlow

Proof-gated cross-chain payments for autonomous agents

Natural language for intent.
Attestcoin for truth.
Deterministic contracts for money.

5 gates · 62 live checks · 32 tests · 2 exact matches

AttestFlow

Source ↗

62 LIVE CHECKS PASSING

One payment. Two chains. No trusted payment oracle.

AttestFlow turns a natural-language settlement rule into a proof-bound CC3 action, then carries the exact on-chain result back to the source chain. Every hash below is public, final on testnet and re-verifiable.

The first use case is cross-border freelance escrow: the client pays where liquidity already exists, while Creditcoin releases only after the exact external payment is cryptographically proven.

ETHEREUM SEPOLIA01

A real client payment succeeds

0.01 native ETH · status 1 · payer and payee independently verified

0x6ac68ba9_59d555e7 ↗

CREDITCOIN CC302

The ASC proves, checks and settles

Attestcoin verify + proof-derived index + receipt status + replay guard

0xec29d5b4_89c2e4c2 ↗

ETHEREUM SEPOLIA03

The exact published payload executes

Same payload hash · authorized signature · destination replay guard

0xc692a176...796947fb ↗

SECURITY INVARIANTS

8 / 8

What the verifier proves

Proof inclusion	BlockProver verify() accepted on CC3
Index binding	calculateTxIndex() = Sepolia receipt index 69
Source success	Attested receipt status = 1
Policy match	Payee and 0.01 ETH value match the rule

INTEGRITY ANCHOR

Published payload hash

0x4845f5ca486987ddb30d486e58f36ed0cebbf5e514d20783d220b06f0d523faa

SourceTxId0x780a2c16...2f28fd1a

Released0.001 CTC

Destination0x83A0b8D2...194f868E

ATTESTFLOW · BUIDL CTC 2026 FALL

01

Autonomous payments have a truth problem

**An AI agent can understand a payment rule.
It cannot be trusted to assert that the payment happened.**

The usability layer may be probabilistic. The truth and money paths must be cryptographic and deterministic.

01 Centralized indexers

A service assertion becomes the settlement trigger.

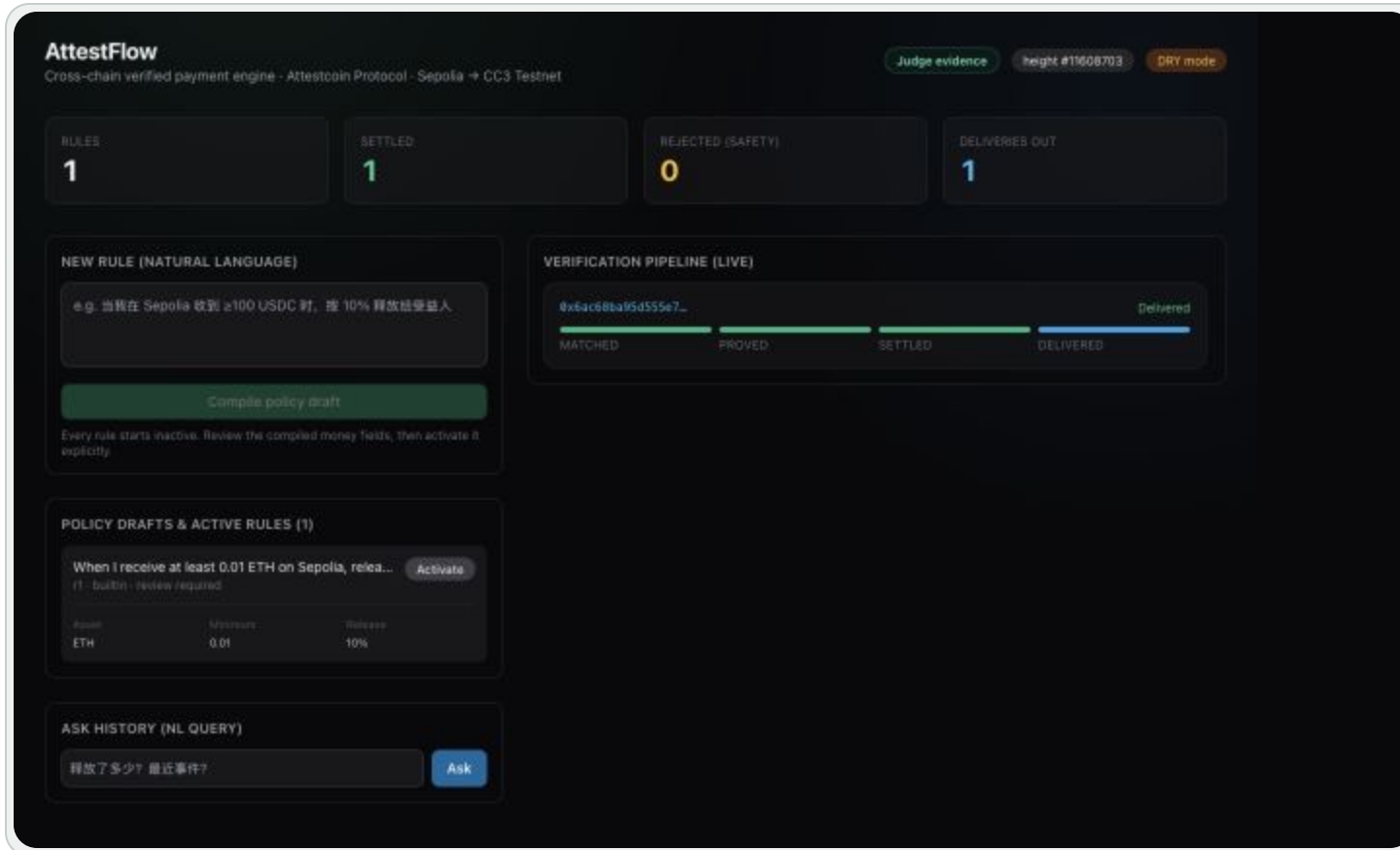
02 Bridge operators

Custody or relayer trust replaces source-chain truth.

03 Unchecked LLM output

A fluent answer can still be financially false.

Natural language never reaches money unchecked



1

Intent becomes a draft

Exact money fields stay inactive until explicit activation.

2

Only attested blocks scan

Discovery cannot race ahead of Creditcoin attestation.

3

Proof gates settlement

The ASC—not the model—decides whether funds can move.

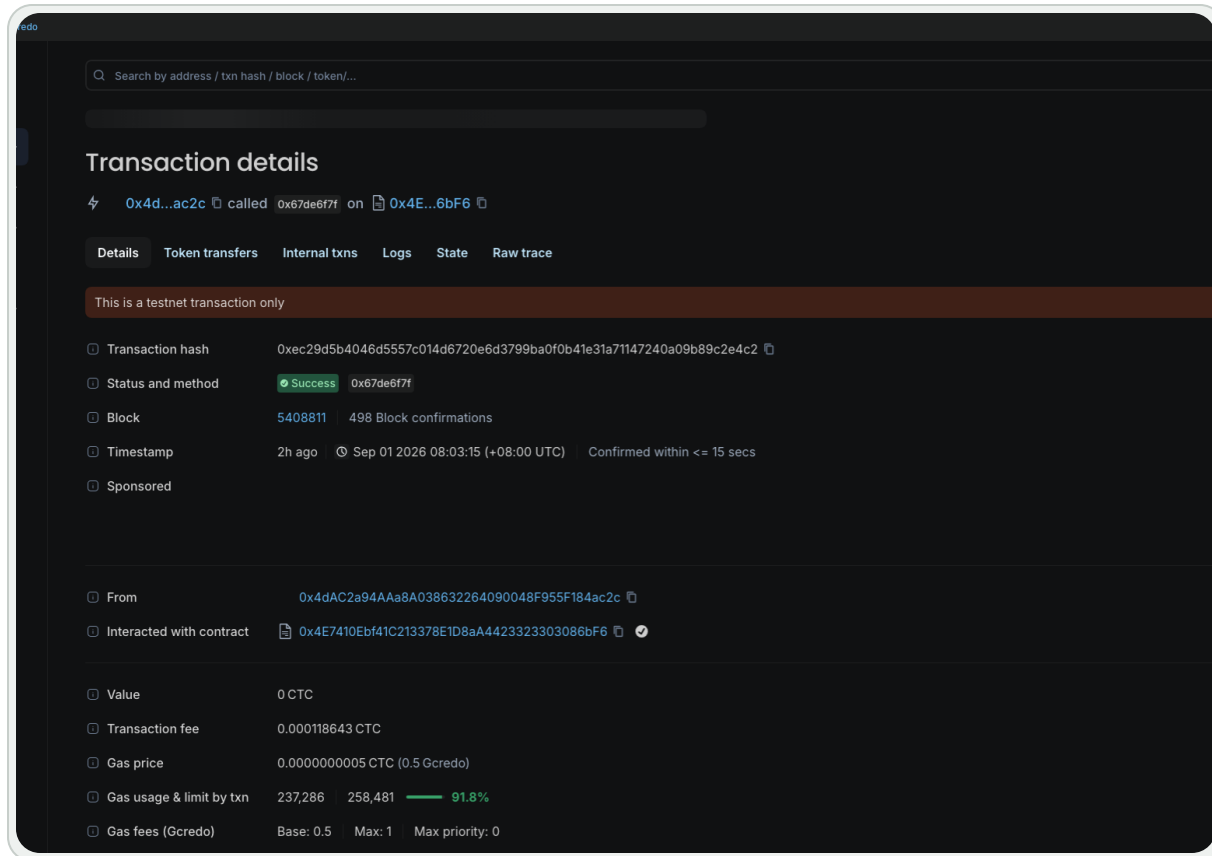
4

The exact result returns

Published bytes execute behind a second replay guard.

Optional LLM for parsing and Q&A. Mandatory deterministic validation at every financial boundary.

Attestcoin is the trust root—not a logo in the stack



- 01 **ChainInfo · 0x0FD3**
Sepolia chainId 11155111 must resolve to chainKey 1.
- 02 **ProofBuilder**
Builds single and shared multi-block proofs.
- 03 **BlockProver · 0x0FD2**
verify() gates settlement; verifyBatch() gates backlogs.
- 04 **calculateTxIndex()**
Derives the replay identity from the Merkle path.
- 05 **Encoding v1**
Exposes transaction fields and receipt status to the ASC.

LIVE ASC **0x4E7410Eb41C213378E1D8aA4423323303086bF6**

The ASC closes the gaps the precompile leaves open

ETHEREUM SEPOLIA 01

A real client payment succeeds
0.01 native ETH · status 1 · payer and payee independently verified
0x6ac68ba9..59d555e7 ↗

CREDITCOIN CC3 02

The ASC proves, checks and settles
Attestcoin verify + proof-derived index + receipt status + replay guard
0xec29d5b4..89c2e4c2 ↗

ETHEREUM SEPOLIA 03

The exact published payload executes
Same payload hash · authorized signature · destination replay guard
0xc692a176..796947fb ↗

SECURITY INVARIANTS 8 / 8

What the verifier proves

Proof inclusion	BlockProver verify() accepted on CC3
Index binding	calculateTxindex() = Sepolia receipt index 69
Source success	Attested receipt status = 1
Policy match	Payee and 0.01 ETH value match the rule
Replay safety	ASC and Inbox replay guards are both set
Payload integrity	CC3 publish hash = Sepolia execute hash
Code identity	Deployed runtime bytecode = local Solidity build
Crash recovery	Destination leg resumed without a second settlement

INTEGRITY ANCHOR

Published payload hash
0x4845f5ca486987ddb30d486e58f36ed0cebbf5e514d28783d220b06f0d523faa

SourceTxId	0x780a2c16..2f28fd1a
Released	0.001 CTC
Destination	0x83A0b802..194f868E

HONEST BOUNDARY
Official Writability contracts are not live on this testnet. The return leg mirrors publish → sign → deliver → validate with one authorized relayer; it does not claim attester-quorum security.

REPRODUCE THE VERDICT

```

npm ci
npm run ci
npm run verify:evidence
# - { "status": "SUCCESS", "checks": 62 }

```

BLOCKPROVER PROVES

Transaction inclusion + source-chain continuity

THE ASC ALSO ENFORCES

- Proof-derived transaction index
- Receipt status == 1
- Recipient · asset · calldata · amount
- Stored policy · beneficiary · destination
- Source replay guard · escrow solvency

A failed source transaction—or an operator-invented replay key—cannot settle.

One payment. Three linked public-testnet legs.

The source payment, proof-gated settlement and destination execution are public—and tied by proof-derived identity plus one payload hash.

1

ETHEREUM SEPOLIA

0.01 ETH received

Source tx
0x6ac68b...55e7
block 11,608,703 · index 69
receipt status = 1

2

CREDITCOIN CC3

0.001 CTC released

Settlement
0xec29d5...e4c2
proof + policy + escrow + replay
checks

3

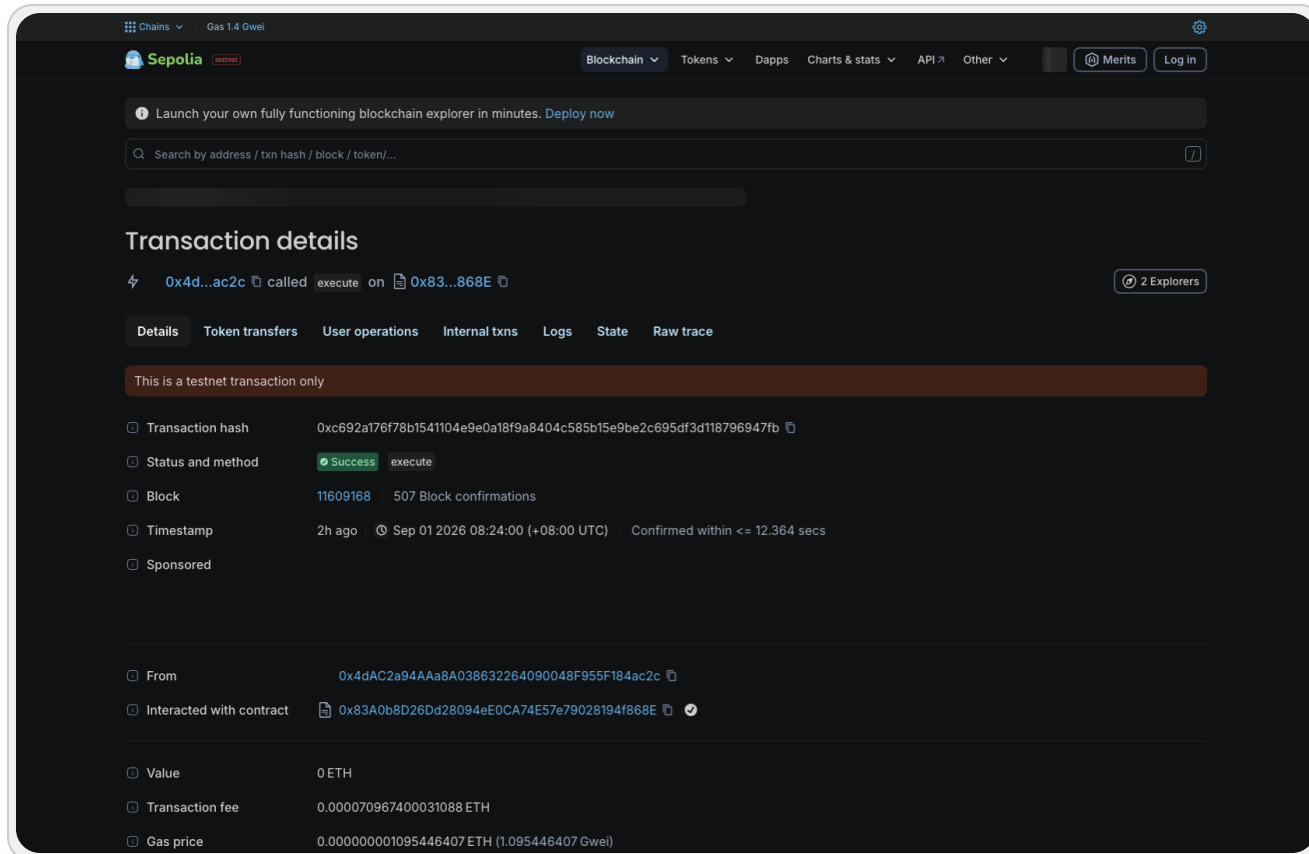
ETHEREUM SEPOLIA

Payload executed

Destination tx
0xc692a1...47fb
Inbox signature + destination replay
guard

PAYLOAD HASH 0x4845f5ca486987ddb30d486e58f36ed0cebbf5e514d20783d220b06f0d523faa

The return leg is honest—and crash-resumable



CURRENT TESTNET BOUNDARY

Official Outbox/Inbox contracts are not deployed here. The adapter uses one authorized relayer—not attestor-quorum security.

RECOVERY EXERCISED IN THE LIVE RUN

- 01 CC3 settlement finalized
- 02 Sepolia RPC timed out
- 03 Restart recovered MessagePublished from chain logs
- 04 **Only the missing destination leg completed**

Published payload hash = executed payload hash

A judge can reproduce the verdict with one command

5

independent judge gates

62

live cross-chain checks

2

exact source matches

32 tests · production build · 0 vulnerabilities ·
chain evidence · compiler identity · fresh
batch + single proofs

attestflow – five independent gates

```
$ npm run judge:verify
```

```
1/5  CI + 32 tests + build + audit      SUCCESS
2/5  62 public-chain checks             SUCCESS
3/5  2 Sourcify exact matches           SUCCESS
4/5  3-block batch · verifyBatch()      SUCCESS
5/5  fresh single · verifySingle()      SUCCESS
```

```
{ "status": "SUCCESS" }
```


Built for CEIP—not a one-off demo

Freelancer escrow is the wedge. The product is a reusable proof-gated payment engine for invoices, marketplaces, treasuries and agent-to-agent commerce.

NOW

Deployed testnet loop

Sepolia → Attestcoin → CC3 settlement
→ Sepolia return

NEXT

Quorum-backed Writability

Swap the adapter for official
Outbox/Inbox when available

SCALE

Payment policy infrastructure

Multi-chain assets, invoice workflows
and agent commerce

Natural language for intent. Attestcoin for truth. Deterministic contracts for money.

`github.com/ximilu0114-droid/ximilu-hackson`